

Assignment 1 - Database Design for ReadVerse Online Bookstore

HTNO: 2503A52L16

1. Identify and list any three types of structured data:

The three types of structured data that should be stored in the ReadVerse bookstore system are:

1. Customer Data – includes Customer ID, Name, Email, Phone, Address.
2. Book Data – includes Book ID, Title, Genre, ISBN, Price, Author ID.
3. Order Data – includes Order ID, Customer ID, Book ID, Order Date, Quantity, Payment Status.

2. Suitable Type of Database:

A Relational Database Management System (RDBMS) such as MySQL, PostgreSQL, or Oracle is most suitable.

Reason:

- It provides structured storage using tables, ensuring data integrity through constraints.
- It supports SQL queries for easy retrieval and analysis of data.
- It is best for handling relationships between customers, books, authors, and orders.

3. Customers Table Design:

Essential Columns:

- CustomerID (Primary Key): Unique identifier for each customer.
- Name: Full name of the customer.
- Email: Contact email of the customer.
- Phone: Customer's phone number.
- Address: Customer's delivery address.

SQL Query to create Customers Table:

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100) NOT NULL,  
    Email VARCHAR2(100) UNIQUE NOT NULL,  
    Phone VARCHAR2(15),  
    Address VARCHAR2(255)  
);
```

```
SQL> desc Customers;
```

Name	Null?	Type
CUSTOMERID	NOT NULL	NUMBER
NAME	NOT NULL	VARCHAR2(100)
EMAIL	NOT NULL	VARCHAR2(100)
PHONE		VARCHAR2(15)
ADDRESS		VARCHAR2(255)

4. Books and Authors Table Design:

Important Columns for Books Table:

- BookID (Primary Key): Unique identifier for each book.
- Title: Title of the book.
- Genre: Category of the book.
- ISBN: International Standard Book Number.
- Price: Price of the book.
- AuthorID (Foreign Key): References Authors table.

Important Columns for Authors Table:

- AuthorID (Primary Key): Unique identifier for each author.
- Name: Full name of the author.
- Bio: Short biography of the author.

SQL Queries to create Books and Authors Tables:

```
CREATE TABLE Authors (  
    AuthorID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100) NOT NULL,  
    Bio CLOB  
);
```

```
SQL> desc Authors;
```

Name	Null?	Type
AUTHORID	NOT NULL	NUMBER
NAME	NOT NULL	VARCHAR2(100)
BIO		CLOB

```
CREATE TABLE Books (  
    BookID NUMBER PRIMARY KEY,  
    Title VARCHAR2(150) NOT NULL,  
    Genre VARCHAR2(50),
```

ISBN VARCHAR2(20) UNIQUE,
Price NUMBER(10,2) NOT NULL,
AuthorID NUMBER,
FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
);

```
SQL> desc Books;
```

Name	Null?	Type
BOOKID	NOT NULL	NUMBER
TITLE	NOT NULL	VARCHAR2(150)
GENRE		VARCHAR2(50)
ISBN		VARCHAR2(20)
PRICE	NOT NULL	NUMBER(10,2)
AUTHORID		NUMBER

SQL Queries to create Orders and Stores Tables:

CREATE TABLE Orders (

OrderID NUMBER PRIMARY KEY,

CustomerID NUMBER,

BookID NUMBER,

OrderDate DATE,

Quantity NUMBER,

PaymentStatus VARCHAR2(50),

FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),

FOREIGN KEY (BookID) REFERENCES Books(BookID)

);

```
SQL> desc Orders;
```

Name	Null?	Type
ORDERID	NOT NULL	NUMBER
CUSTOMERID		NUMBER
BOOKID		NUMBER
ORDERDATE		DATE
QUANTITY		NUMBER
PAYMENTSTATUS		VARCHAR2(50)
STOREID		NUMBER

CREATE TABLE Stores (

StoreID NUMBER PRIMARY KEY,

StoreName VARCHAR2(100) NOT NULL,

Location VARCHAR2(150) NOT NULL

);

```
SQL> desc Stores;
Name                                         Null?    Type
-----
STOREID                                     NOT NULL NUMBER
STORENAME                                  NOT NULL VARCHAR2(100)
LOCATION                                    NOT NULL VARCHAR2(150)
```

5. Use of Joins to Connect Data:

Joins can be used to connect Customers, Books, and Authors to generate meaningful reports.
Example: To find which customers bought books from a particular author.

SQL Query:

(i) Show customers with authors of purchased books

SELECT c.Name AS CustomerName, a.Name AS AuthorName

FROM Customers c

JOIN Orders o ON c.CustomerID = o.CustomerID

JOIN Books b ON o.BookID = b.BookID

JOIN Authors a ON b.AuthorID = a.AuthorID;

```
SQL> SELECT c.Name AS CustomerName, a.Name AS AuthorName
2  FROM Customers c
3  JOIN Orders o ON c.CustomerID = o.CustomerID
4  JOIN Books b ON o.BookID = b.BookID
5  JOIN Authors a ON b.AuthorID = a.AuthorID;

CUSTOMERNAME                                AUTHORNAME
-----
Rahul Sharma                                J.K. Rowling
Rahul Sharma                                Chetan Bhagat
Sneha Reddy                                 J.K. Rowling
Amit Verma                                  George R.R. Martin
```

(ii) Show customers with the bookstore location

```
SELECT c.Name AS CustomerName, s.Location AS StoreLocation  
  
FROM Customers c  
  
JOIN Orders o ON c.CustomerID = o.CustomerID  
  
JOIN Stores s ON o.StoreID = s.StoreID;
```

```
SQL> SELECT c.Name AS CustomerName, s.Location AS StoreLocation  
2 FROM Customers c  
3 JOIN Orders o ON c.CustomerID = o.CustomerID  
4 JOIN Stores s ON o.StoreID = s.StoreID;
```

CUSTOMERNAME	STORELOCATION
Rahul Sharma	Hyderabad
Sneha Reddy	Bangalore
Amit Verma	Hyderabad
Rahul Sharma	Bangalore

(iii) Show customers with authors and bookstore location

```
SELECT c.Name AS CustomerName, a.Name AS AuthorName, s.Location AS  
StoreLocation  
  
FROM Customers c  
  
JOIN Orders o ON c.CustomerID = o.CustomerID  
  
JOIN Books b ON o.BookID = b.BookID  
  
JOIN Authors a ON b.AuthorID = a.AuthorID  
  
JOIN Stores s ON o.StoreID = s.StoreID;
```

```
SQL> SELECT c.Name AS CustomerName, a.Name AS AuthorName, s.Location AS StoreLocation  
2 FROM Customers c  
3 JOIN Orders o ON c.CustomerID = o.CustomerID  
4 JOIN Books b ON o.BookID = b.BookID  
5 JOIN Authors a ON b.AuthorID = a.AuthorID  
6 JOIN Stores s ON o.StoreID = s.StoreID;  
rows will be truncated
```

CUSTOMERNAME	AUTHORNAME
Rahul Sharma	Chetan Bhagat
Sneha Reddy	J.K. Rowling
Rahul Sharma	J.K. Rowling
Amit Verma	George R.R. Martin

(iv) Find all customers who bought books written by “Chetan Bhagat” and from which store

```
SELECT c.Name AS CustomerName, b.Title AS BookTitle, s.Location AS  
StoreLocation  
  
FROM Customers c  
  
JOIN Orders o ON c.CustomerID = o.CustomerID
```

JOIN Books b ON o.BookID = b.BookID

JOIN Authors a ON b.AuthorID = a.AuthorID

JOIN Stores s ON o.StoreID = s.StoreID

WHERE a.Name = 'Chetan Bhagat';

```
SQL> SELECT c.Name AS CustomerName, b.Title AS BookTitle, s.Location AS StoreLocation
2 FROM Customers c
3 JOIN Orders o ON c.CustomerID = o.CustomerID
4 JOIN Books b ON o.BookID = b.BookID
5 JOIN Authors a ON b.AuthorID = a.AuthorID
6 JOIN Stores s ON o.StoreID = s.StoreID
7 WHERE a.Name = 'Chetan Bhagat';
rows will be truncated
```

CUSTOMERNAME	BOOKTITLE
Rahul Sharma	2 States